

Arquitetura em Camadas — AdoteApp

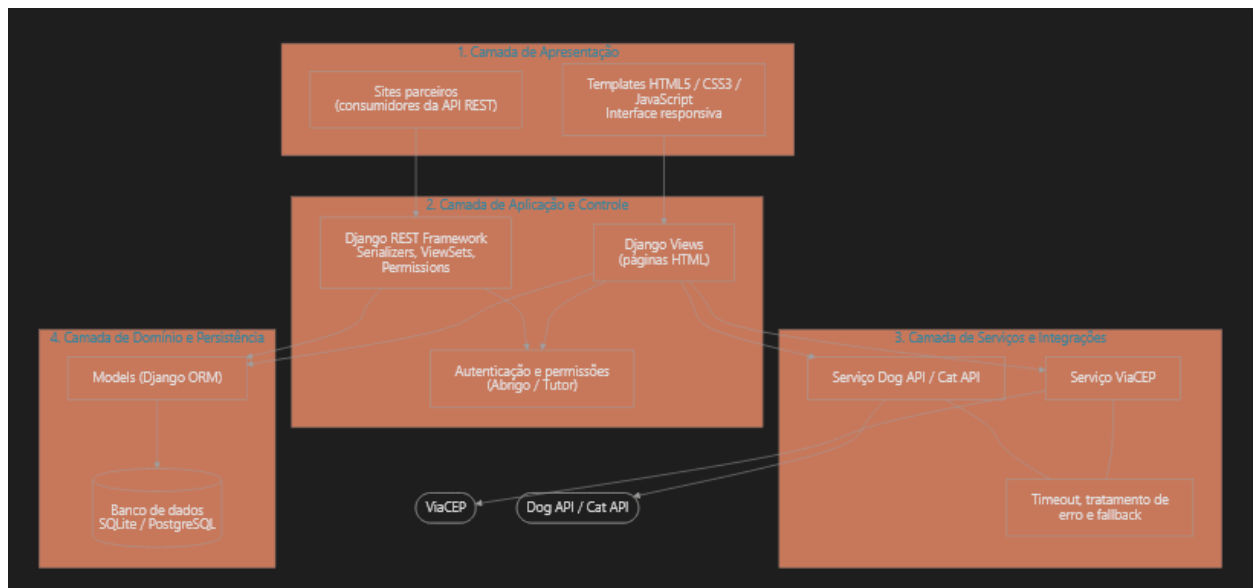
Projeto	AdoteApp — Plataforma de Adoção e Cuidados de Animais
Disciplina	Desenvolvimento Web — ADS (CEUB)
Professor	Felippe Pires Ferreira
Data	25/09/2026
Repositório	https://github.com/Bruhbs-123/Projeto-web

1. Visão geral

O **AdoteApp** adota uma arquitetura em camadas baseada no padrão **MVT (Model-View-Template)** do framework Django, combinada com uma arquitetura **RESTful** (Django REST Framework) para disponibilizar dados a aplicações externas, como sites parceiros.

A separação em camadas traz coesão ao código, facilita a manutenção, permite reaproveitar regras de negócio e **isola a comunicação com APIs de terceiros** (ViaCEP, Dog API e Cat API), de modo que uma falha externa não derrube o restante do sistema.

Diagrama 1 — Camadas e componentes da aplicação



2. Detalhamento das camadas

2.1 Camada de Apresentação

Responsável pela interface com o usuário e pela exibição das informações.

- **Templates Django:** páginas HTML5 dinâmicas, com componentes reutilizáveis (cabeçalho, rodapé, modais e formulários).
- **Interface responsiva:** estilização adaptada a computadores e dispositivos móveis, seguindo a identidade visual do projeto.
- **Feedback ao usuário:** JavaScript assíncrono (Fetch API) para o preenchimento automático de endereço pelo CEP e para a exibição de sugestões de raça sem recarregar a página. Essas chamadas são feitas **a endpoints internos do PetMatch**, que por sua vez consultam as APIs externas; o navegador não se comunica diretamente com a ViaCEP nem com a Dog/Cat API.
- **Consumidores da API:** sites parceiros acessam os dados pela API REST pública, sem passar pelos templates.

2.2 Camada de Aplicação e Controle

Recebe as requisições da camada de apresentação, aplica validações de segurança e executa a lógica de negócio.

- **Django Views / ViewSets:** processam requisições HTTP (**GET**, **POST**, **PUT**, **DELETE**), validam o estado das entidades e retornam HTML ou JSON.

- **Serializers (DRF):** convertem objetos e *QuerySets* em JSON para a API pública (GET /api/animais e GET /api/animais/{id}) e validam os dados de entrada nas operações restritas.
- **Autenticação e permissões:** gerenciam sessões, controlam o acesso por perfil (Abrigo ou Tutor/Adotante) e armazenam senhas com hash.

2.3 Camada de Domínio e Persistência

Representa as entidades do sistema, seus relacionamentos e regras de integridade.

- **Django ORM:** mapeia classes Python para tabelas do banco relacional e gerencia as migrations.
- **Modelos principais:**

Modelo	Responsabilidade
Usuario / Perfil	Dados de autenticação, perfil de acesso e vínculo com abrigo ou tutor
Abrigo e Tutor	Dados cadastrais e endereço (preenchido via ViaCEP) [CONFIRMAR com o DER]
Animal	Nome, espécie, raça, porte, idade, foto, localização (CEP) e status: Disponível, Em processo, Adotado, Inativo
Adocao	Vínculo e histórico entre o animal, o abrigo doador e o adotante
RegistroSaude	Vacinas e consultas: datas de aplicação, descrição e controle da próxima dose (reforço)

Os nomes dos modelos devem coincidir com o DER e com o código (`models.py`). Se no código as vacinas e consultas forem modelos separados (`Vacina` e `Consulta`), ajustar a última linha da tabela.

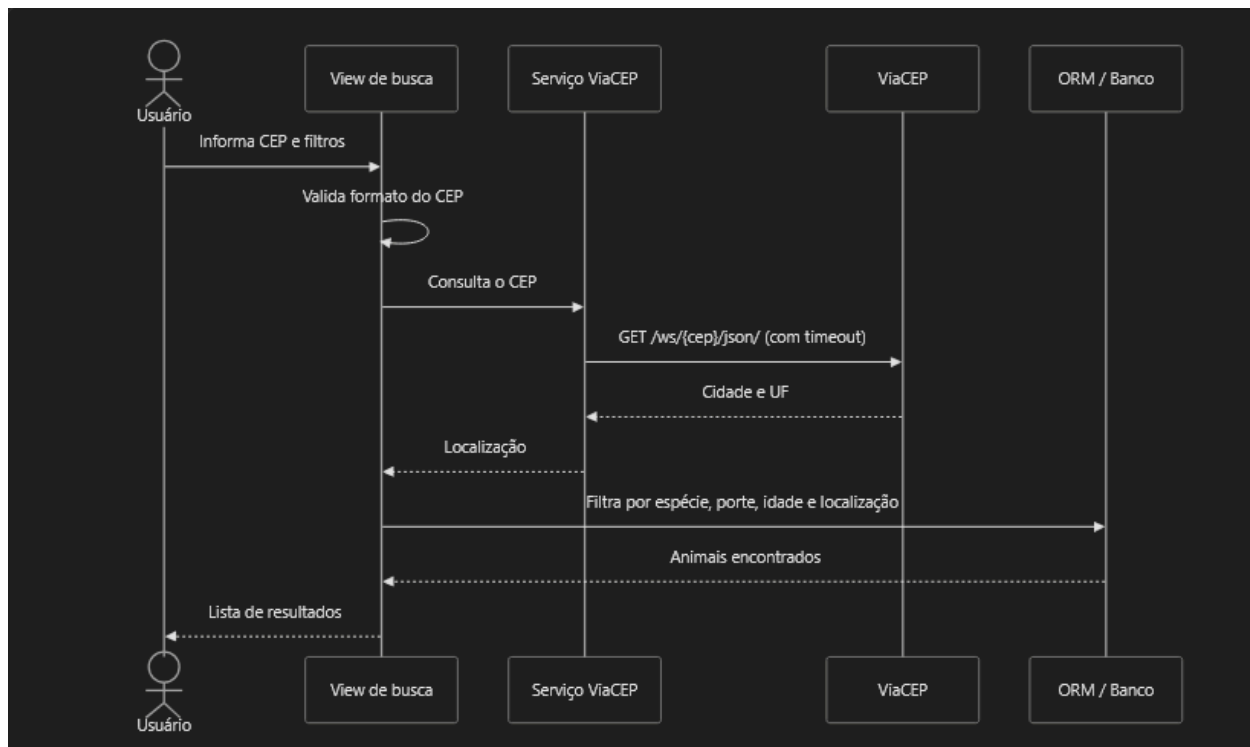
2.4 Camada de Serviços e Integrações Externas

Módulo isolado que concentra a comunicação HTTP com serviços de terceiros, garantindo resiliência.

- **Serviço ViaCEP:** consulta logradouro, bairro, cidade e UF a partir de um CEP de 8 dígitos. Os dados são usados no cadastro de abrigos e tutores e na busca por localização.
- **Serviço Dog API / Cat API:** consulta raças caninas e felinas para obter fotos de referência e características, usadas no cadastro do animal.
- **Resiliência e tolerância a falhas:**
 - *timeouts* explícitos para evitar que requisições fiquem travadas;
 - tratamento de exceções para erros de rede, respostas inválidas e erros HTTP 5xx;
 - respeito aos limites de requisição de cada serviço;
 - chaves de acesso lidas de variáveis de ambiente, nunca do código;
 - **fallback:** se um serviço externo estiver indisponível, a aplicação exibe uma mensagem clara e permite preencher manualmente endereço, raça e foto, mantendo o cadastro utilizável.

3. Fluxo de dados

Diagrama 2 — Busca de animais por CEP



Se a ViaCEP falhar, a view avisa o usuário e oferece a busca sem localização.

Outros fluxos principais

- **Cadastro de animal:** o abrigo preenche o formulário, o sistema consulta a Dog/Cat API para sugerir raça e foto, valida os dados e grava o animal no banco.
 - **API pública:** o cliente externo chama `GET /api/animais` com filtros, as permissões são verificadas, o serializer monta a resposta e o JSON paginado é devolvido.
 - **Vacinas pendentes:** a view consulta os registros de saúde com próxima dose futura ou vencida e monta o calendário e o relatório.
-

4. Tecnologias

Finalidade	Tecnologia
Linguagem e framework	Python 3.13 e Django
API REST	Django REST Framework
Documentação da API	Swagger/Redoc [CONFIRMAR biblioteca]
Banco de dados	SQLite (desenvolvimento) e PostgreSQL (produção) [CONFIRMAR]
Front-end	HTML5, CSS3, JavaScript e Bootstrap [CONFIRMAR]
Integrações HTTP	<code>requests</code> com timeout [CONFIRMAR]
Segurança	Bandit/Semgrep (SAST) e OWASP ZAP (DAST)
Publicação	Servidor de aplicação (ex.: gunicorn), HTTPS, variáveis de ambiente [CONFIRMAR]

5. Implantação e segurança

Em produção, a aplicação roda com `DEBUG=False`, `ALLOWED_HOSTS` e `CSRF_TRUSTED_ORIGINS` restritivos e conexão HTTPS. Segredos (chave do Django, chave da

Dog/Cat API e credenciais do banco) são injetados por variáveis de ambiente. Arquivos `.env` e bancos locais não são versionados.

6. Decisões arquiteturais

Decisão	Justificativa
Monólito Django em camadas	Menor complexidade e adequado ao prazo e ao tamanho da equipe
Camada de serviços para APIs externas	Isola falhas, facilita testes e permite trocar o provedor
Chamadas externas feitas só pelo servidor	Evita expor chaves, permite controlar timeout e limites
DRF para a API pública	Serialização, paginação, filtros e permissões prontos
Controle de acesso por perfil	Protege as operações de escrita e os dados de cada usuário

7. Rastreabilidade

Camada	Requisitos atendidos	Casos de uso
Apresentação	RF06, RF07, RF08	Buscar animais, visualizar relatórios
Controle	RF01, RF02, RF05, RF10, RF11	Cadastrar animal, registrar adoção, consultar API
Serviços	RF03, RF09	Preencher endereço, importar raça e foto
Persistência	RF01 a RF05	Todos